

Ensemble Learning

Arun Chauhan, GEU Dehradun

Ensemble Learning

Bagging • Random Forest • Boosting • Stacking

Central question:

Can many imperfect models together make a better model?

Why Do We Need Ensembles?

- A single model may be sensitive to the training data.
- Different models can make different errors.
- Combining models can improve generalization.
- Three key ideas: reduce variance, reduce bias, or learn how to combine models.

The Basic Idea

Single model

$$X \rightarrow Model \rightarrow \hat{y}$$

Ensemble

$$X \rightarrow \begin{cases} M_1 \\ M_2 \\ M_3 \\ \vdots \\ M_K \end{cases} \rightarrow \text{Combination} \rightarrow \hat{y}$$

The fundamental idea:

Combine diverse models to improve generalization

Why Can Ensembles Work?

There are three important sources of error:

Bias + Variance + Noise

Ensemble methods primarily try to address:

Bagging

Reduce Variance

Boosting

Reduce Bias

Stacking

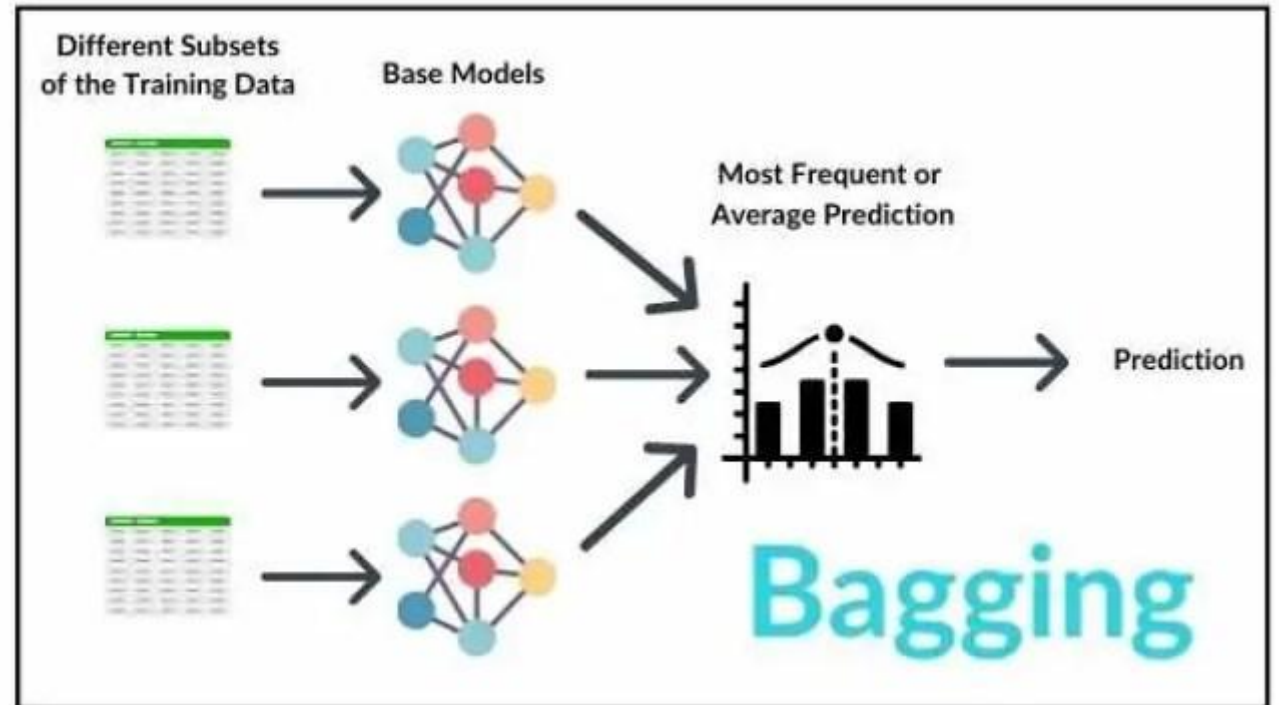
Learn how to combine different models

Three Major Ensemble Strategies

Method	Main idea	Training
Bagging	Train models independently on different samples	Parallel
Boosting	Train models sequentially, focusing on mistakes	Sequential
Stacking	Train different models and learn a meta-model	Two levels

Bagging = Bootstrap Aggregating

- Create multiple bootstrap datasets by sampling WITH replacement.
- Train one model on each bootstrap sample.
- **Classification:** majority vote.
- **Regression:** average predictions.
- Main objective: **reduce variance.**



Bagging: Simple Example

Original data: 1 2 3 4 5 6

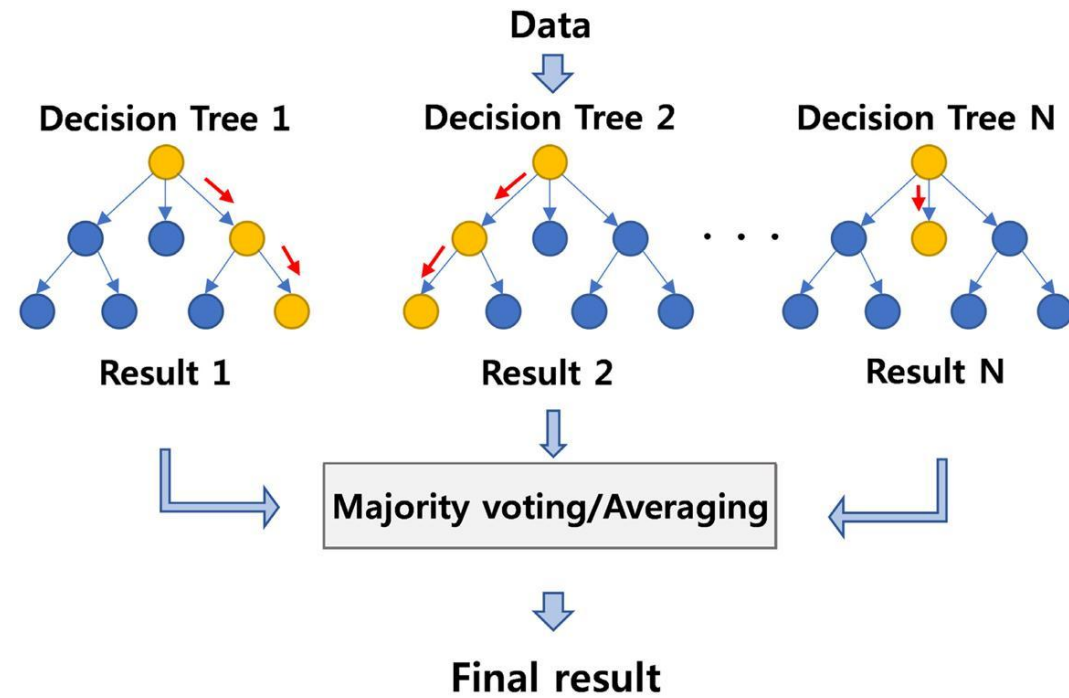
Bootstrap samples may contain repeated observations.

$D_1 = 1\ 2\ 2\ 4\ 6\ 6$

$D_2 = 1\ 3\ 4\ 4\ 5\ 6$

$D_3 = 2\ 2\ 3\ 5\ 5\ 6$

Train Tree 1, Tree 2, Tree 3, ... independently.



Why Does Bagging Help?

Bagging and Variance

For intuition, suppose every model has variance:

$$\sigma^2$$

If predictions were independent, averaging B models gives approximately:

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{B}$$

Therefore:

$$B \uparrow \Rightarrow \text{Variance} \downarrow$$

But there is a catch:

■ The trees are not completely independent.

This leads naturally to **Random Forest**.

Random Forest

Random Forest combines two kinds of randomness:

1. Bootstrap sampling of training examples.
2. Random selection of features at each split.

Each tree is grown using a different sample and different candidate features.

Random Forest Intuition

Suppose we have:

100 features

At a particular tree node, instead of considering all 100:

randomly select m features

For example:

$$m = 10$$

and choose the best split only among those 10.

Different trees therefore see different:

- training examples
- features

which makes trees **less correlated**.

Why Random Feature Selection?

Suppose all trees use the same strongest feature.

Then:

Tree 1 
Tree 2 
Tree 3  very similar predictions
Tree 4 
Tree 5 

They make similar errors.

Random feature selection produces:

Tree 1 → different
Tree 2 → different
Tree 3 → different
Tree 4 → different
Tree 5 → different

Hence:

Lower correlation $\Rightarrow$ better ensemble

Random Forest: Algorithm

Training

For $b = 1, \dots, B$:

1. Draw bootstrap sample D_b
2. Grow a decision tree
3. At every node:
 - randomly select m features
 - find best split among those m
4. Continue until stopping criterion.

Prediction

Classification:

$$\hat{y} = \text{mode}(T_1(x), \dots, T_B(x))$$

Regression:

$$\hat{y} = \frac{1}{B} \sum_b T_b(x)$$

Bagging vs Random Forest

Property	Bagging	Random Forest
Bootstrap samples	✓	✓
Multiple trees	✓	✓
Random features	Usually no	✓
Main purpose	Reduce variance	Reduce variance + decorrelate trees
Base model	Any unstable learner	Decision trees
Parallel training	✓	✓

Boosting

Bagging asks:

| "Can I train many models independently?"

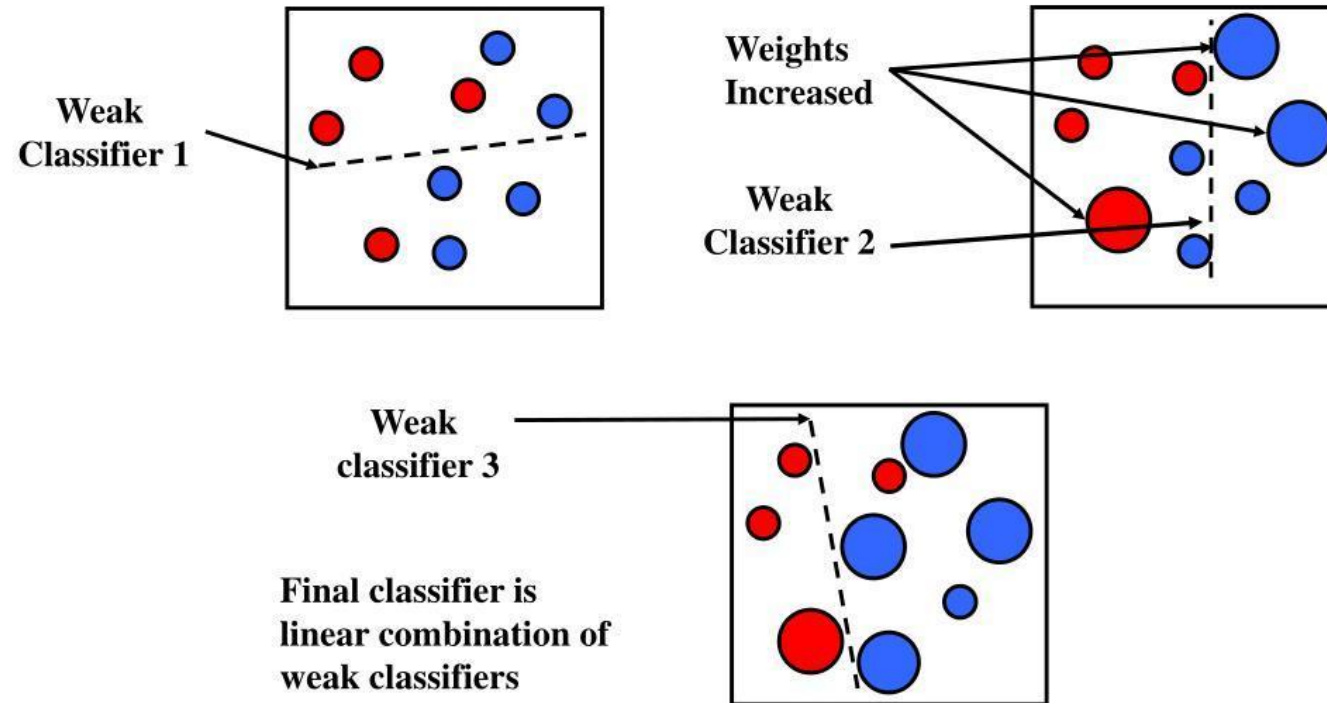
Boosting asks:

| "Can I make the next model focus on what the previous model got wrong?"

This is the central idea.

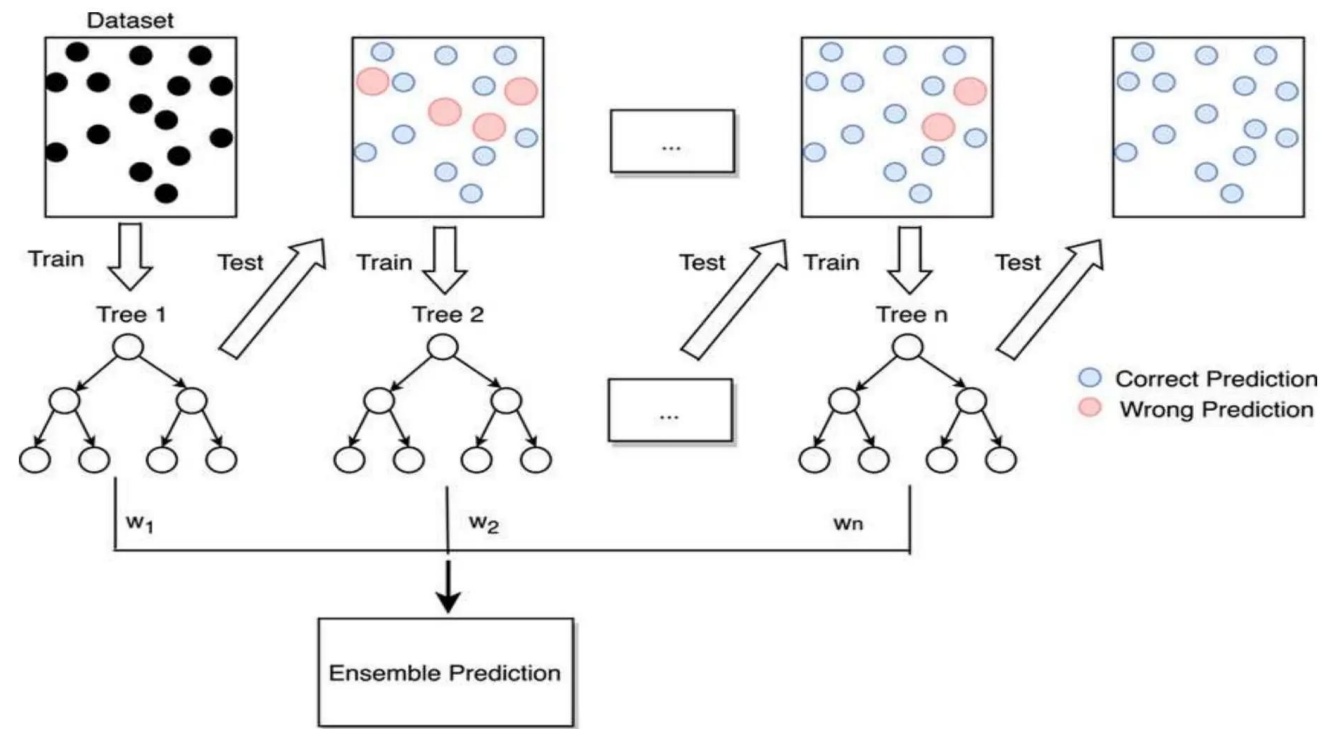
Boosting: Intuition

Updating the weight of examples



Boosting

Flow Diagram of Boosting



AdaBoost:

Step 1 — Initial Weights

- Suppose there are N training examples.
- Initially every example is equally important :

$$w_i = \frac{1}{N}$$

- Example with 6 observations:

each weight = 1/6.

Step 2 — Train Weak Learner

- Train a weak learner $h_1(x)$, often a decision stump.
- Calculate its weighted error:

$$\epsilon_t = \sum_i w_i I(y_i \neq h_t(x_i))$$

- Only mistakes contribute to the weighted error.

Step 3 — Weight the Learner

- Learner weight:
$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$
- Lower error $\rightarrow$ larger α .
- A learner performing better than random guessing receives positive influence.

Step 4 — Increase Weight of Mistakes

$$w_i \leftarrow w_i \exp(-\alpha_t y_i h_t(x_i))$$

- Correctly classified example: its weight decreases.
- Misclassified example: its weight increases.
- Conceptually:
- Wrong example $\rightarrow$ higher weight $\rightarrow$ next learner pays more attention.
- Normalize the weights so that

$$\sum_i w_i = 1$$

AdaBoost: The Classroom Picture

- **Round 1:** Everyone gets equal attention.
- **Round 2:** Find the examples Model 1 got wrong.
- **Round 3:** Increase their weights.
- **Round 4:** Train Model 2 with greater attention on difficult examples.
- Repeat until enough weak learners are trained.

AdaBoost: Final Prediction

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

- Each weak learner votes.
- But better learners have larger α_t and therefore more influence.
- This is weighted voting, not simple majority voting.

Stacking

Instead of:

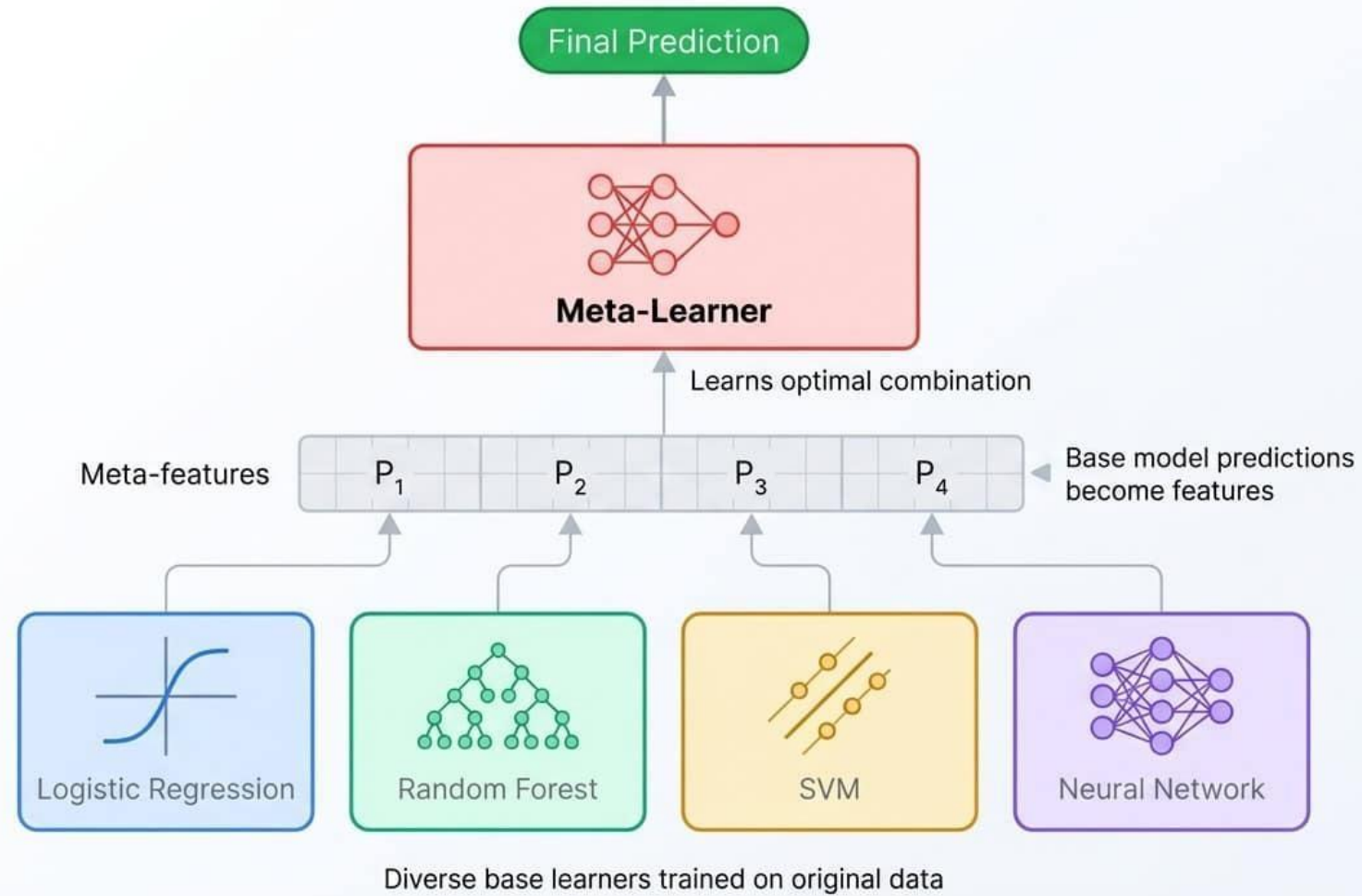
- "Vote among models."

we ask:

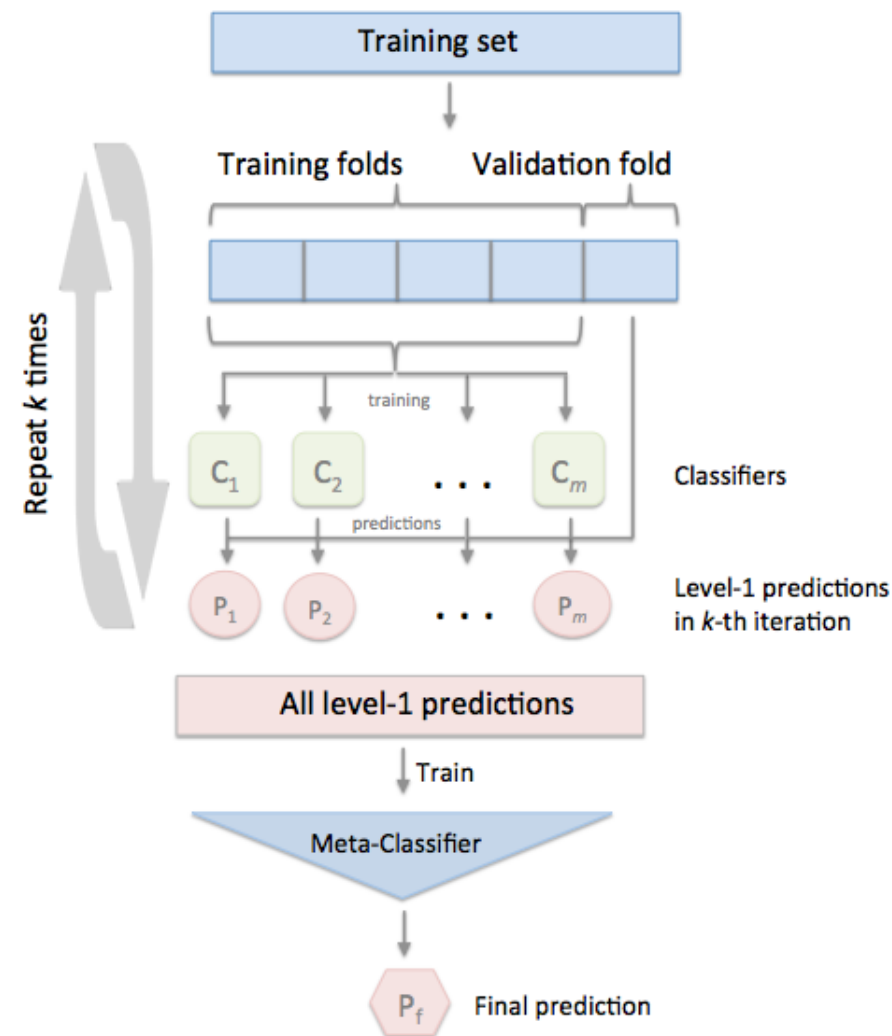
- "Can another model learn how to combine their predictions?"**

This is **Stacking**.

Stacking Architecture



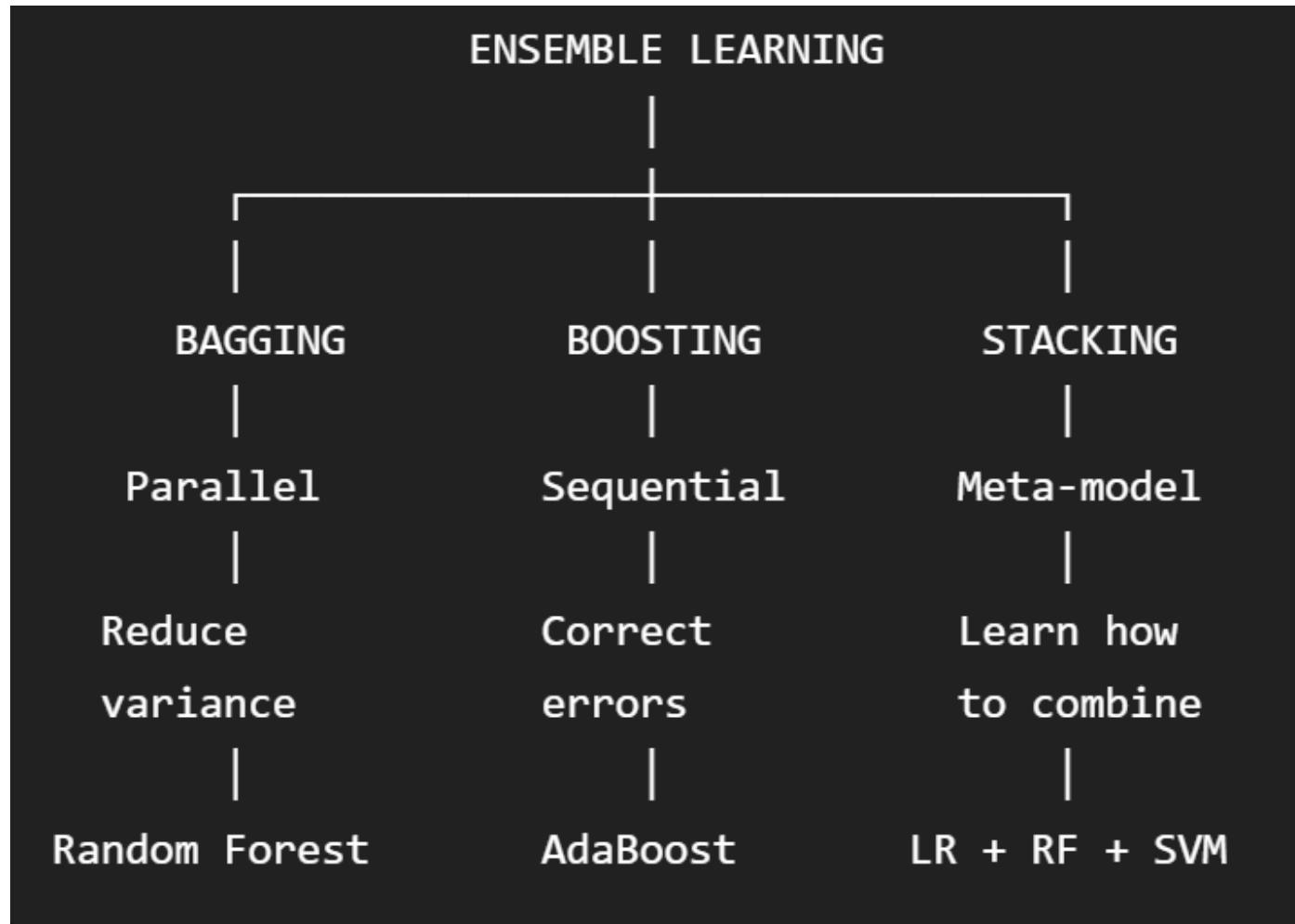
Cross-Validation in Stacking



Why Cross-Validation Matters in Stacking

- If the meta-model trains on predictions from models that saw the same examples, leakage can occur.
- Use out-of-fold (OOF) predictions for training the meta-model.
- Each training example gets a prediction from a base model that did NOT train on that example.
- Then train the meta-model on these OOF predictions.

The Big Picture



When Should We Use What?

- Need a robust tabular-data baseline → Random Forest.
- Unstable base learner / want variance reduction → Bagging.
- Want strong predictive performance with sequential refinement → Boosting.
- Different algorithms have complementary strengths → Stacking.
- Always validate with appropriate cross-validation and held-out test data.

Key References

Breiman, L. (1996). Bagging Predictors. *Machine Learning*, 24, 123–140.

Breiman, L. (2001). Random Forests. *Machine Learning*, 45, 5–32.

Freund, Y. & Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting.

Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, 5(2), 241–259.

Schapire et al. (1998). Boosting the Margin: A New Explanation for the Effectiveness of Voting Methods.

End — Quick Check

1. Why does Bagging reduce variance?
2. What extra randomness does Random Forest introduce?
3. Why is Boosting sequential?
4. Why do misclassified examples receive more attention?
5. What is the role of the meta-model in Stacking?
6. Why are out-of-fold predictions important?

Questions?